Three-Way Word Embeddings for Polingual: Script, Sound and Meaning, and a Re-Ranked Lexicon

Gianangelo Dichio
Bucket Foundation

2026-11-06

Abstract

Polingual translates single words offline from a Wiktionary lexicon for 28 languages. The lexicon lists English meanings in English frequency order, so a rare sense can come first: Hebrew *akhil* (edible) was the first translation of English *good*. We embed 529,131 words of 29 languages four ways: rendered script pixels with a small CNN, character n-grams, phonetic features from Wiktionary IPA with a grapheme model for words without IPA, and LaBSE meaning vectors, plus a joint projection of all four [empirical: dataset v1 manifest.json, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/vocab.py, then python3 apple/scripts/embed/meaning.py, then python3 apple/scripts/embed/store.py]. Meaning vectors then re-rank every translation pair, with gloss senses added as candidates. On the held-out half of an independent Tatoeba-aligned word set, the first English sense shown is right for 68.3% of words, up from 58.0% [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. For English to target words, a model audit of 100 changed first answers finds 91 right after re-ranking and 60 before [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. A secondary any-token score in the app evaluation, which includes the tuning rows and rewards the longer answers, rises from 67.5% to 77.2% [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport]. The shipped change is the re-ranked lexicon, 0.24 MB larger [empirical: du -cb apple/Shared/Corpus/lexicon/*.tsv.z before and after, 2026-11-06].

1 Introduction

A word lookup in Polingual returns up to three English senses from the bundled lexicon. For the lexicon of pull request 1198, any-token precision was 98.7% on the Wiktionary reference row and 67.5% on a word set aligned from Tatoeba sentence pairs [empirical: data/swift_before.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport]. The gap comes from sense order: Wiktionary translation tables list a foreign word under every English headword that uses it, and the builder kept the four most frequent English headwords. A foreign word that translates a rare sense of a frequent English word inherits that word.

The founder asked for three separate embeddings per word, of script, sound and meaning, and a joint one, and for higher accuracy. This paper builds the four, measures each against a task it should solve, and uses them to score every translation pair in the lexicon.

2 Related work

Word vectors and cross-lingual maps. Skip-gram and CBOW learn word vectors from co-occurrence [1, 2], and subword n-grams extend them to unseen spellings [3]. A linear map between two monolingual spaces translates words by nearest neighbour [4]. Constraining the map to be orthogonal turns it into the orthogonal Procrustes problem, which has a closed-form solution through the singular value decomposition [5, 6, 7, 8]. VecMap learns the map with almost no seed dictionary by self-learning [9], and MUSE learns it with no dictionary by adversarial training and scores neighbours with cross-domain similarity local scaling to reduce hubness [10, 11]. A survey covers the family [12].

Evaluating lexicon induction. Precision at k against a gold dictionary is the standard score, and several studies question it. Glavaš et al. show that lexicon induction scores do not predict how well a cross-lingual space serves downstream tasks [13]. Kementchedjhieva et al. find that the MUSE dictionaries are dominated by proper nouns and miss valid translations, so a correct answer can count as wrong [14]. Czarnowska et al. show that scores fall on rare and inflected forms [15]. Mapping methods assume that two spaces are close to isomorphic. The assumption fails for many language pairs [16], and how far it fails depends on training data and budget as well as on typology [17]. Our gold set inherits the first problem, Section 7.

Multilingual encoders. Sentence encoders trained on translation pairs place translations near each other without any map: LASER with a shared BiLSTM [18], LaBSE with a dual BERT encoder [19], and Sentence-BERT models distilled into many languages [20, 21]. We encode bare words with LaBSE.

Sources. Wiktionary is a collaboratively written dictionary, and its coverage has been compared with expert-built lexicons [22]; Wiktextract parses it into the machine-readable files that kaikki.org serves [23]. WikiPron scrapes its IPA transcriptions, and its data defined the SIGMORPHON 2020 grapheme-to-phoneme task [24, 25]. Tatoeba sentence pairs give test sets for 112 languages [18, 26]; FLORES-101 gives professionally translated sentences for 101 languages [27], at sentence level only, so we align words from Tatoeba.

Form-based views. Rendered text as model input handles noise and unseen scripts in translation and language modelling [28, 29], and CNNs over character images learn the visual composition of Chinese, Japanese and Korean characters [30], with glyph vectors for Chinese in particular [31]. Phonetic word embeddings built from articulatory features have their own benchmark [32], and phonetic features help find cognates across languages [33]. Contrastive objectives with in-batch negatives [34, 35] train both our script encoder and our joint projection. PanPhon, Mortensen et al., COLING 2016, `https://aclanthology.org/C16-1328/`, and the WikiPron paper have no DOI or arXiv identifier, so they stay out of the bibliography and are named in the text; WikiPron data is cited above.

3 Preliminaries

Definition 3.1 (Lexicon)**.** For a language L, the lexicon maps a headword w to an ordered list $E(w)$ of English words and a gloss $g(w)$, a list of short English sense strings from the word's own Wiktionary entry. The app shows the first three items of $E(w)$, or of $g(w)$ when $E(w)$ is

empty. For English word e the lexicon also holds an ordered list $F(e)$ of target words; English-to-target lookups show the first. Lean: `Lexicon`, `Lexicon.shown`, `Lexicon.firstTarget`, with `Lexicon.shown_length_le` proving at most three items show, Appendix B.

Definition 3.2 (Modality)**.** A modality m is a map from words to unit vectors in $\mathbb{R}^{d_m}$. The similarity of words a, b under m is the dot product $\cos_m(a, b)$ of their unit vectors. Lean: `Modality` and `Modality.cos`, with `Modality.cos_symm` proving symmetry, Appendix B.

Definition 3.3 (Joint score)**.** Given weights w_m, the joint score of a pair is $s(a, b) = \sum_m w_m \cos_m(a, b)$. With weight one on meaning and zero elsewhere it equals the meaning cosine, proved as `jointScore_meaning_only` in Appendix B; `jointOf` builds the score from a list of modalities and `jointOf_symm` proves it symmetric.

Definition 3.4 (Precision)**.** For an evaluation row with J judged answers of which R are right, precision is R/J in percent, `precisionPct` in Lean; `precisionPct_le` in Appendix B proves it stays within the percent scale when $R \leq J$. First-sense precision judges only the text before the first semicolon of the answer; any-token precision judges the whole answer.

4 Method

All embedding and scoring ran on host charizard, AMD Radeon RX 7600 with ROCm 6.4 and torch 2.9.1 [36]; the re-ranker ran on its CPU with the GPU hidden. Each tag below names the report file in dataset v1, the first eight hex digits of its sha256 from `manifest.json`, and the command.

Vocabulary. The lexicon comes from Wiktionary through the Wiktextract files on kaikki.org [23, 22]. The lexicon builder of pull request 1198 runs in a dump mode, `python3 apple/scripts/translate/build_lexdl out/base 900000 repo/apple/Shared/Corpus - out/candidates.json`, that writes, for every shipped headword, all English candidates in frequency order and the gloss, and for every English headword all target candidates. Gloss senses that equal an English headword add target candidates to its list. The vocabulary is every headword, every candidate and every evaluation word: 26,887 English words and 502,244 words across the 28 target languages [empirical: dataset v1 manifest.json, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/vocab.py, then python3 apple/scripts/embed/meaning.py, then python3 apple/scripts/embed/store.py].

Script. Each word is rendered in Noto Sans for Latin, Greek and Cyrillic, Noto Sans Arabic, Hebrew, Devanagari, Tamil and Thai, and Noto Sans CJK for Chinese, Japanese and Korean, with HarfBuzz shaping through Pillow's Raqm layout, into a 32×128 grey raster, following visual text representations [28, 29]. A four-block CNN maps the raster to 64 dimensions [30]. It trains for 2,000 steps of batch 512 on 116,000 words, 4,000 per language, rendered at font weights 300, 400 and 700; two views of a word with random scale, shear, shift and noise form a positive pair under a symmetric InfoNCE loss [34, 35] with temperature 0.07 [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script.py]. The character view, after fastText [3], hashes the word's 1 to 3-grams with boundary marks into 2^{20} buckets and sums a fixed Gaussian vector per bucket, 64 dimensions [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script.py].

Phonetic. WikiPron files `CUNY-CL/wikipron/data/scrape/tsv` give space-separated IPA scraped from Wiktionary [24, 25]; we use up to 150,000 entries per language, 1,589,580 in all [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py]. Each segment maps to the 24 articulatory features of PanPhon, Mortensen et al., COLING 2016, `https://aclanthology.org/C16-1328/`, code `dmort27/panphon`, in $\{-1, 0, 1\}$, the feature type PWESuite benchmarks [32]; the word vector averages the segment features inside four equal position bins, giving 96 dimensions that keep coarse order [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py]. For words without IPA a grapheme model, a regression form of grapheme-to-phoneme conversion [24], predicts the vector: the word splits into units, extended grapheme clusters by default, Hangul jamo for Korean, pinyin initials and finals for Chinese, kana with small-kana merging for Japanese; each unit, each unit at its position bin, each unit bigram and the first and last unit index a learned 96-d vector in an EmbeddingBag, and the mean of those vectors regresses the IPA vector under squared error for 8 epochs [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py]. The unit vectors are the letter and grapheme-cluster level of the phonetic embedding. 313,678 of the 529,131 vocabulary rows carry IPA; the grapheme model fills the rest [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py].

Meaning. LaBSE, `sentence-transformers/LaBSE` at revision `836121a0`, encodes each bare word to 768 dimensions through sentence-transformers [19, 20]. Gloss strings are encoded the same way, with parenthesised notes removed [empirical: dataset v1 manifest.json, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/vocab.py, then python3 apple/scripts/embed/meaning.py, then python3 apple/scripts/embed/store.py].

Joint. Two joint forms are fitted on translation pairs from 80% of each language's headwords, split by hash [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py], and the other 20% are test queries [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]. The weighted form searches $w_m \in \{0, 0.05, 0.1, 0.2, 0.3\}$ for script, character and phonetic with meaning fixed at one, maximising P@1 on 300 training queries per language; it selects 0.05 for script and zero for the other two [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]. A logistic regression on the four cosines gave a phonetic weight of -2.05 and was dropped [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]: sound similarity between a foreign word and its English translation carries signal only for cognates. The projection form scales each modality, concatenates the 992 dimensions and maps them linearly to 256, a learned linear map in the manner of [4], trained for 4,000 steps of batch 1,024 under symmetric InfoNCE [34] at temperature 0.05 with duplicate English targets masked [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py].

Re-ranking. Re-ranking scores each Wiktionary candidate by cosine in the meaning space, the retrieval rule of lexicon induction [10, 13], restricted to candidates the dictionary already lists. For headword w, candidates are $E(w)$ in frequency order j plus up to three gloss senses at gloss position j. Candidate x scores $\cos(w, x) - \beta \log(1 + j)$, plus a bonus b for the first gloss sense, and

the top four ship. For English e, target candidate t scores $\cos(e,t) - \beta_r \log(1+j) + \gamma\,\pi(t)$ with $\pi(t) = 1 - \log(1+r_t)/\log(60001)$ from the wordfreq rank r_t of t in its language, zero when unranked; a candidate spelled the same as e drops to the end. The scorer, β, a drop threshold, a relative cut, gloss use and b were chosen on the even rows of the Tatoeba-aligned word set by first-sense precision with coverage held at the baseline [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py], and $\beta_r \in \{0, 0.05, 0.1\}$ and $\gamma \in \{0, 0.1, 0.2, 0.3, 0.5, 0.75, 1, 1.5, 2, 3\}$ on the even half of 378 common English words with Tatoeba-derived targets [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. Selected: meaning cosine, $\beta = 0.05$, no drop threshold, gloss on, $b = 0.1$, $\beta_r = 0.1$, $\gamma = 0.75$, an interior optimum of the γ grid [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. The odd rows and odd words are held out. The builder applies the English-to-target order only when `REV_RERANK=1`, the default, set by the audit in Section 5.2; the shipped lexicon was built with `REV_RERANK=1`, and its Hebrew line for *good* now starts with *tov*: `REV_RERANK=1 python3 apple/scripts/translate/build_lexicon.py dl out/rerank 900000 repo/apple/Shared/Corpus out/order.json`.

English-to-target references. For each English word and language, target tokens are scored by Dice co-occurrence [37] with the English token over Tatoeba pairs [26], character 1 to 3-grams for Chinese, Japanese and Thai, and the top three with count at least three are the references [empirical: dataset v1 reports/regression_report.json sha256 64a4ec7a, 2026-11-06, charizard CPU: python3 apple/scripts/embed/regression.py]. A first choice is right when it equals a reference, shares its first four letters with one, or for the unspaced languages contains or is contained in one.

5 Results

5.1 Re-ranked lexicon

Table 1: Held-out odd rows of the Tatoeba-aligned word set, 2,325 words, Python replica of the app lookup [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py].

Precision, percent	before	after
First shown sense	58.0	68.3
Any shown sense, longer answers	67.3	78.5

On the held-out half, first-sense precision rises from 58.0% to 68.3% [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. The mean answer grew from 5.35 to 8.58 words, because gloss senses are phrases [empirical: dataset v1 out/rerank lexicon, 2026-11-06, charizard CPU: python3 apple/scripts/embed/answer_length.py]; any-token precision, which counts a hit anywhere in that longer answer, rises from 67.3% to 78.5% on the same rows [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py].

Table 2: Held-out odd rows per language, percent: first-sense precision, then any-token precision [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py].

		first sense			any token	
Lang	n	before	after	change	before	after
es	100	69.0	78.0	+9.0	76.0	89.0
fr	100	64.0	83.0	+19.0	70.0	92.0
it	100	66.0	72.0	+6.0	76.0	87.0
pt	100	67.0	79.0	+12.0	74.0	88.0
de	100	63.0	74.0	+11.0	76.0	90.0
nl	100	77.0	82.0	+5.0	81.0	91.0
sv	100	59.0	66.0	+7.0	70.0	80.0
pl	100	59.0	74.0	+15.0	71.0	86.0
fi	100	60.0	75.0	+15.0	68.0	85.0
ru	100	60.0	73.0	+13.0	69.0	81.0
el	100	63.0	73.0	+10.0	69.0	81.0
ar	100	49.0	61.0	+12.0	65.0	69.0
hi	34	47.1	47.1	+0.0	50.0	50.0
ja	100	49.0	59.0	+10.0	59.0	71.0
zh	100	50.0	61.2	+11.2	58.2	66.3
ko	88	36.4	39.8	+3.4	47.7	51.1
he	100	50.0	58.0	+8.0	62.0	71.0
fa	100	54.0	68.0	+14.0	65.0	76.0
tr	100	51.0	69.0	+18.0	59.0	74.0
cs	100	74.0	82.0	+8.0	83.0	89.0
id	100	60.0	66.0	+6.0	72.0	76.0
vi	100	36.0	48.0	+12.0	45.0	58.0
th	100	57.0	64.0	+7.0	68.0	74.0
la	100	63.0	71.0	+8.0	68.0	87.0
sa	3	0.0	33.3	+33.3	33.3	33.3

Table 3: Model audit of English-to-target first answers, 100 items drawn by hash from the 1,488 of 4,848 cases where the two orders differ. Judged by Claude Opus 5.5 against the common Wiktionary sense and Tatoeba usage; this is a model audit, and no human checked it [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py].

Order	items	right
Frequency order	100	60
Re-ranked	100	91

5.2 English-to-target audit

Re-ranking changes the first English-to-target answer for 1,488 of 4,848 common or polysemous English words across the languages [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. In 100 of them the frequency order was right 60 times and the re-ranked order 91 times [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. The 100 split into 34 wins, 3 losses, 57 right under both orders and 6 wrong under both [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. All 3 losses are new errors the re-rank introduced: *why* to Spanish moves from *por qué* to *por*, *inside* to Japanese from *naka ni* to the loanword *in*, and *hot* to Korean from *deopda* to the loanword *hat* [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. The 6 wrong under both include *pitch* to German, *right* to Czech and *give* to Spanish [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py]. Typical wins: *wind* to Finnish moves from *kiemurrella* (to meander) to *tuuli*, *root* to Czech from *odmocnina* (square root) to *kořen*. A human check of 30 items, the 3 losses, the 6 shared errors, 12 wins, 4 shared successes and 5 unchanged answers, is written to `audit/human-spotcheck.tsv` with empty verdict columns; it is pending, and until it returns the audit numbers are model judgments. Against Tatoeba-derived references on the held-out half of the common English words, first-choice accuracy rises from 60.6% to 68.0% [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. The English-to-target Wiktionary row of the app evaluation falls from 94.7% to 89.9% [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport], because its references are the frequency-ordered translation lists themselves.

5.3 Polysemous case study

The ten words good, right, light, fair, well, kind, mean, bank, play and run were added after the founder saw *good* become *akhil* in Hebrew. They were not held out from the design of the English-to-target prior, so they are a case study: the first choice matches a Tatoeba-derived reference in 109 of 158 language cases after re-ranking and 81 before [empirical: dataset v1 reports/regression_report.json sha256 64a4ec7a, 2026-11-06, charizard CPU: python3 apple/scripts/embed/regression.py], and *good* to Hebrew now gives *tov*.

5.4 App evaluation

In the app evaluation the Tatoeba word row rises from 67.5% to 77.2%, phrase precision from 14.1% to 17.2% and sentence precision from 7.7% to 9.9% [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport]; the Wiktionary row stays at 98.7% [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport]. These rows include the even rows used for tuning and reward longer answers. The CI floor test passes. On the iPhone 17 Pro simulator, a Debug build loads and parses one lexicon in 664 to 2,106 ms off the main thread, and six loads through the four-slot cache peak at 67.1 MB of process footprint [empirical: data/lexicon_bench.txt, 2026-11-06, Mac: TEST_RUNNER_LEXICON_BENCH=1 xcodebuild test -scheme BucketKit -destination iPhone 17 Pro simulator -only-testing lexiconLoadBench].

Table 4: App evaluation, `TranslateEvalTests`, target word to English, all rows including the tuning rows, any-token precision in percent. Secondary to Table 2 [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport].

	Tatoeba word row			Wiktionary row	
Lang	before	after	change	before	after
es	74.5	88.0	+13.5	99.6	99.6
fr	74.5	87.0	+12.5	99.5	99.5
it	75.5	88.0	+12.5	99.0	99.0
pt	76.5	89.5	+13.0	99.5	99.5
de	76.5	88.5	+12.0	98.9	98.9
nl	80.0	90.0	+10.0	99.7	99.7
sv	73.5	84.0	+10.5	98.4	98.5
pl	68.5	82.5	+14.0	98.5	98.5
fi	67.0	82.5	+15.5	99.4	99.4
ru	68.0	79.5	+11.5	99.9	99.9
el	73.5	82.0	+8.5	99.2	99.2
ar	62.5	66.0	+3.5	100.0	100.0
hi	52.9	55.9	+3.0	99.3	99.3
ja	59.0	71.5	+12.5	100.0	100.0
zh	64.1	68.7	+4.6	99.8	99.8
ko	48.9	52.8	+3.9	99.3	99.4
he	63.5	71.0	+7.5	99.4	99.6
fa	65.5	74.5	+9.0	99.9	99.9
tr	60.5	73.0	+12.5	99.5	99.5
cs	81.5	89.5	+8.0	99.8	99.8
id	75.0	79.0	+4.0	99.3	99.3
vi	32.5	39.5	+7.0	81.5	81.5
th	62.5	69.0	+6.5	98.7	98.7
ta	0.0	0.0	+0.0	99.5	99.5
la	72.5	84.5	+12.0	99.7	99.7
sa	42.9	42.9	+0.0	98.9	98.9
all	67.5	77.2	+9.7	98.7	98.7

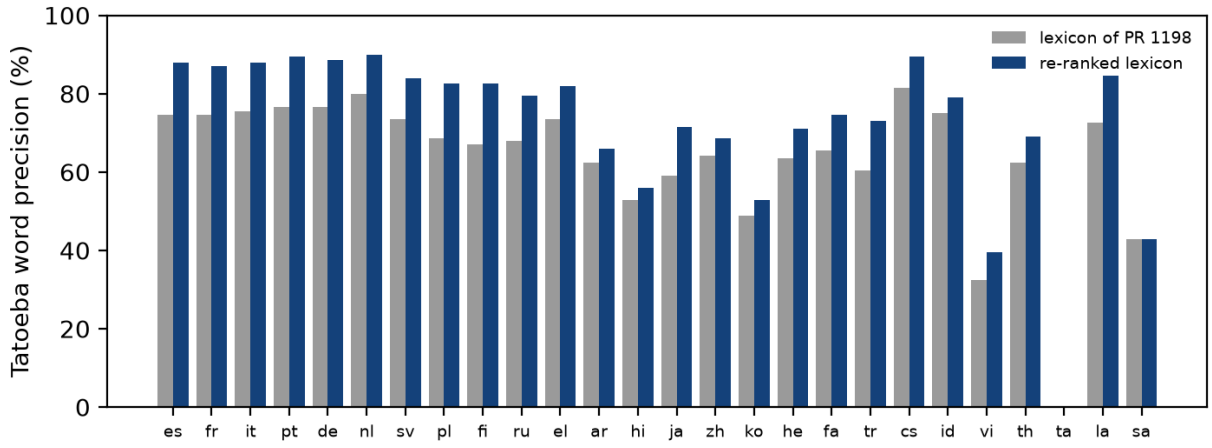


Figure 1: Tatoeba-aligned word row per language in the app evaluation, any-token precision, all rows [empirical: data/swift_before.txt and data/swift_after.txt, 2026-11-06, Mac: TRANSLATE_EVAL=current swift test --package-path BucketKit --filter translateEvalReport].

5.5 Each embedding on its own task

Table 5: Unicode confusable pairs, 924 single letters, ranked among a pool of 4,996 letters [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py].

View	R@1	R@10	MRR
Script CNN	37.2	55.2	0.432
Raw pixels	34.8	48.5	0.397
Character n-grams	0.2	0.2	0.003

The script CNN finds the listed confusable of a letter in its top ten for 55.2% of pairs against 48.5% for raw pixel cosine [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py]; character n-grams cannot, since distinct code points share no n-gram. For fuzzy search with one random edit per word, character n-grams recover the word at rank one in 73.7% of cases on average over 12 languages and the CNN in 52.8% [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py]; their sum reaches 77.2% [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py]. For a swap to a visual confusable the CNN reaches 85.1% and the sum 93.6%, against 77.6% for n-grams [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py].

The phonetic view finds homophones better than character n-grams in 23 of 27 languages with enough pairs, mean R@10 80.2% against 62.3% [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py]. The gain is largest where spelling hides sound: Chinese 81.3% against 6.5%, Korean 87.8% against 7.6%, Japanese 33.7% against 5.6% [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py].

Table 6: Fuzzy search R@1 in percent within each language's vocabulary [empirical: dataset v1 reports/script_report.json sha256 3931c793, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/script_eval.py].

	one random edit			confusable swap		
Lang	CNN	n-gram	sum	CNN	n-gram	sum
es	59.6	72.4	77.4	92.9	75.9	95.8
de	67.1	77.8	82.0	96.0	79.5	98.1
ru	67.1	81.1	85.7	99.4	83.5	99.2
el	64.5	85.3	88.5	97.8	85.1	98.7
ar	28.4	55.8	55.5	44.4	59.2	71.8
he	28.2	57.0	57.7	65.2	63.4	87.2
hi	32.3	67.9	64.6	100.0	96.4	100.0
ta	51.5	77.7	75.2	69.3	81.4	91.1
th	46.6	76.2	77.7	93.3	74.6	94.0
ja	49.3	70.3	81.0	92.0	70.0	96.2
zh	87.9	90.6	97.6	85.4	84.5	97.1
ko	50.9	72.4	83.4	-	-	-

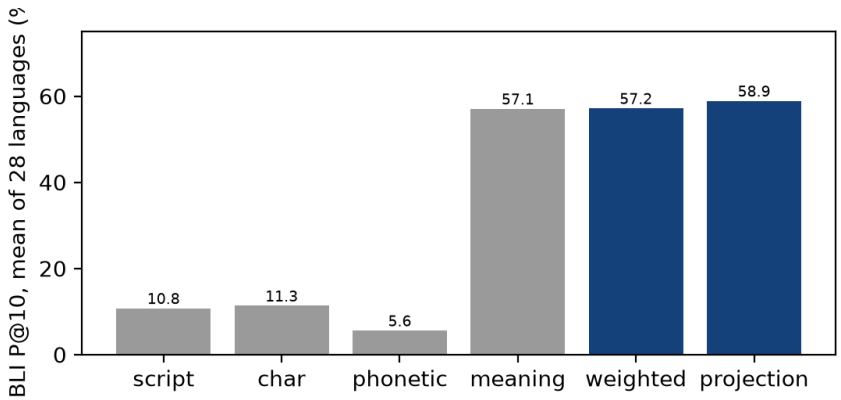


Figure 2: Bilingual lexicon induction, target word to English among 26,887 English words, P@10 averaged over 28 languages, test headwords only [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py].

Table 7: Phonetic view. G2P: the grapheme model on held-out WikiPron words, mean cosine to the IPA vector and R@10 among 3,000. Homophones: distinct spellings with identical IPA, query and pool both predicted from spelling, R@10 in percent for phonetic and character views [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py].

Lang	WikiPron	IPA rows	G2P cos	G2P R@10	pairs		phon	char
en	79306	8198/26887	0.913	57.4	1000	76.5	55.8	
es	131056	15650/22864	0.965	94.8	1000	96.3	85.6	
fr	84402	14929/20813	0.918	65.8	1000	93.8	87.2	
it	81441	14414/19915	0.957	93.1	302	84.4	77.8	
pt	56305	12806/18250	0.944	80.2	977	85.6	71.8	
de	51318	10219/20538	0.923	60.3	671	86.9	75.9	
nl	45234	11852/19425	0.927	61.9	779	84.5	75.2	
sv	5054	3060/22446	0.828	5.5	72	81.9	63.9	
pl	146081	16759/18832	0.949	94.8	1000	92.9	76.3	
fi	150000	14490/21587	0.959	93.0	216	72.2	75.5	
ru	149999	5675/17355	0.954	89.9	1000	99.5	97.3	
el	19008	6367/14964	0.934	57.9	1000	97.1	71.4	
ar	14272	6952/14265	0.908	74.6	97	48.5	76.3	
hi	30379	12189/14862	0.909	66.0	832	73.7	85.5	
ja	44247	11607/18346	0.867	18.4	1000	33.7	5.6	
zh	147316	8052/19348	0.886	72.4	1000	81.3	6.5	
ko	30421	16320/20247	0.927	79.3	370	87.8	7.6	
he	2872	2782/15001	0.821	13.5	45	88.9	42.2	
fa	10728	10253/17162	0.862	31.6	117	85.5	53.8	
tr	11436	8007/20509	0.889	23.1	-	-	-	
cs	63095	21081/22952	0.950	97.3	849	95.5	76.2	
id	17810	11381/17605	0.916	45.9	47	66.0	57.4	
vi	26978	11920/23449	0.893	62.6	527	53.7	23.1	
th	17221	13457/14174	0.845	34.8	589	63.7	55.9	
ta	10095	9345/10873	0.906	57.0	-	-	-	
la	41656	1080/16014	0.946	90.0	496	95.6	90.1	
grc	57540	11293/13020	0.946	80.7	1000	94.4	53.6	
sa	9796	8796/10361	0.894	37.1	30	53.3	56.7	
ang	54514	14744/17067	0.951	95.0	675	93.2	78.8	

Table 8: Bilingual lexicon induction P@10 in percent per language and view; gold is the Wiktionary candidate set [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py].

Lang	n	script	char	phon	meaning	weighted	projection
es	506	26.3	28.5	12.1	68.4	68.4	71.3
fr	540	35.0	38.3	14.1	72.2	72.4	74.3
it	479	25.3	28.6	10.6	71.6	71.8	74.1
pt	454	31.3	31.5	9.5	74.2	74.4	77.5
de	456	20.6	24.3	10.5	65.4	65.6	67.1
nl	477	27.9	27.7	15.5	72.3	72.1	75.7
sv	428	25.5	25.5	5.6	64.0	63.8	64.7
pl	391	16.6	16.9	10.7	76.0	76.5	77.0
fi	542	7.4	8.3	2.4	55.5	55.7	55.7
ru	416	3.1	0.2	4.1	72.6	71.9	73.3
el	698	1.3	0.1	2.6	59.0	59.5	61.7
ar	758	0.1	0.1	0.0	17.8	17.7	23.6
hi	630	0.2	0.2	6.2	59.5	59.2	61.3
ja	359	0.0	0.3	4.5	72.7	72.1	73.5
zh	156	0.0	0.0	1.3	67.9	67.3	67.9
ko	670	0.0	0.0	8.5	61.8	62.2	62.8
he	460	0.0	0.2	0.4	37.0	36.5	38.7
fa	449	0.2	0.2	3.3	40.8	41.2	44.5
tr	681	11.0	12.0	3.1	54.6	54.9	55.2
cs	703	17.8	16.6	7.7	63.0	63.2	67.1
id	421	21.1	23.0	6.9	69.1	69.1	71.0
vi	498	6.6	4.4	0.4	59.2	60.0	60.4
th	418	0.2	0.2	4.3	61.2	62.2	59.6
ta	213	0.0	0.0	0.9	61.0	61.0	63.4
la	510	16.1	18.8	1.2	40.4	41.2	41.8
grc	284	0.0	0.0	2.8	25.0	25.4	26.8
sa	136	0.0	0.0	0.0	37.5	37.5	40.4
ang	148	7.4	10.8	6.8	17.6	17.6	18.2
mean		10.8	11.3	5.6	57.1	57.2	58.9

Meaning carries translation: P@10 57.1% on average, against 10.8% for script, 11.3% for character and 5.6% for phonetic [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]. The joint projection reaches 58.9% P@10 and 36.0% P@1, against 37.1% P@1 for meaning alone [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]; the weighted form matches meaning within 0.1 points [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py]. Across related language pairs, Table 9, the phonetic view is the one form-based view that crosses scripts: Russian to Polish R@10 9.7% against 0.3% for character n-grams [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py].

Table 9: Translation retrieval between related languages, R@10 in percent [empirical: dataset v1 reports/joint_report.json sha256 11b0b40e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/joint.py].

Pair	n	script	char	phon	meaning	weighted	projection
es-pt	417	56.4	55.2	38.8	78.4	78.7	77.0
es-it	411	41.1	36.7	36.7	69.1	70.8	71.5
fr-it	402	31.8	30.3	12.4	68.4	69.7	68.9
de-nl	341	30.5	35.2	27.9	68.3	69.8	67.7
sv-de	350	28.6	31.1	8.6	58.3	60.6	60.3
cs-pl	355	33.5	23.4	25.9	71.0	71.0	69.6
ru-pl	298	3.7	0.3	9.7	76.8	75.8	75.8
hi-sa	257	20.2	26.1	13.6	41.6	40.9	41.6
ar-fa	433	19.2	8.3	3.7	23.1	24.0	27.7
ja-zh	133	30.8	31.6	0.0	66.9	66.9	65.4
el-grc	255	36.1	33.7	19.6	42.4	42.7	45.5
la-it	353	22.1	22.7	2.5	39.9	40.2	41.6

5.6 Language graphs

Figure 3, Figure 4, Figure 5 and Figure 6 are drawn by `papers/tools/langgraph.py`, a matplotlib tool with fixed seeds whose PNGs are byte-identical across runs of `make figures`.

Every script family gains under re-ranking. Hindi is the one language with no first-sense change, 47.1% before and after on 34 held-out words [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py].

The unweighted language mean rises from 55.3% to 66.3% for the first sense and from 65.4% to 75.8% for any token [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard: make figures]; no language falls on either score.

The null baseline matters most for sound: random word pairs already score 0.50 to 0.84 in the phonetic view, against at most 0.13 in the script view, so raw phonetic cosines mostly measure shared phoneme inventories [empirical: data/langgraph.json sha256 e9d1ca42, 2026-11-06, charizard: python3 papers/tools/langgraph.py derive]. On raw phonetic cosines Hebrew, Latin and Greek are hubs with 11, 10 and 10 kept edges; after subtraction the largest degree is 7, for Dutch, and Greek's nearest language by sound is Ancient Greek, the pattern of hubness that nearest-neighbour retrieval shows [11, 10]. Script edges stay mostly inside a script family: 17 of 45 kept edges cross families, and the strongest is Spanish to Portuguese at 0.58 above a null of 0.06 [empirical: data/langgraph.json

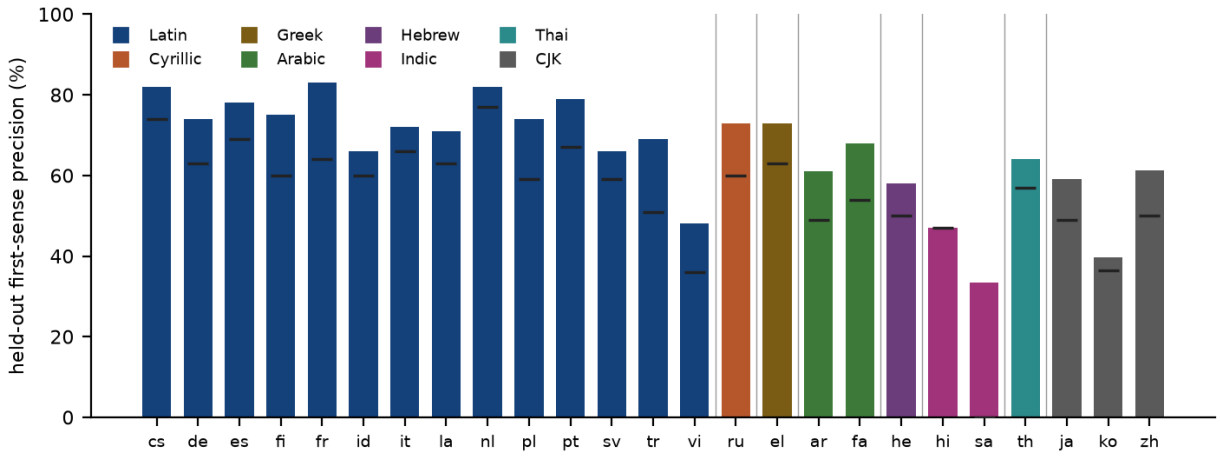


Figure 3: Held-out first-sense precision after re-ranking per language, grouped and coloured by script family; the black tick is the frequency-order baseline. Sanskrit has 3 held-out words and Hindi 34, so their bars carry wide error [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard: make figures, python3 figures/build.py calling papers/tools/langgraph.py].

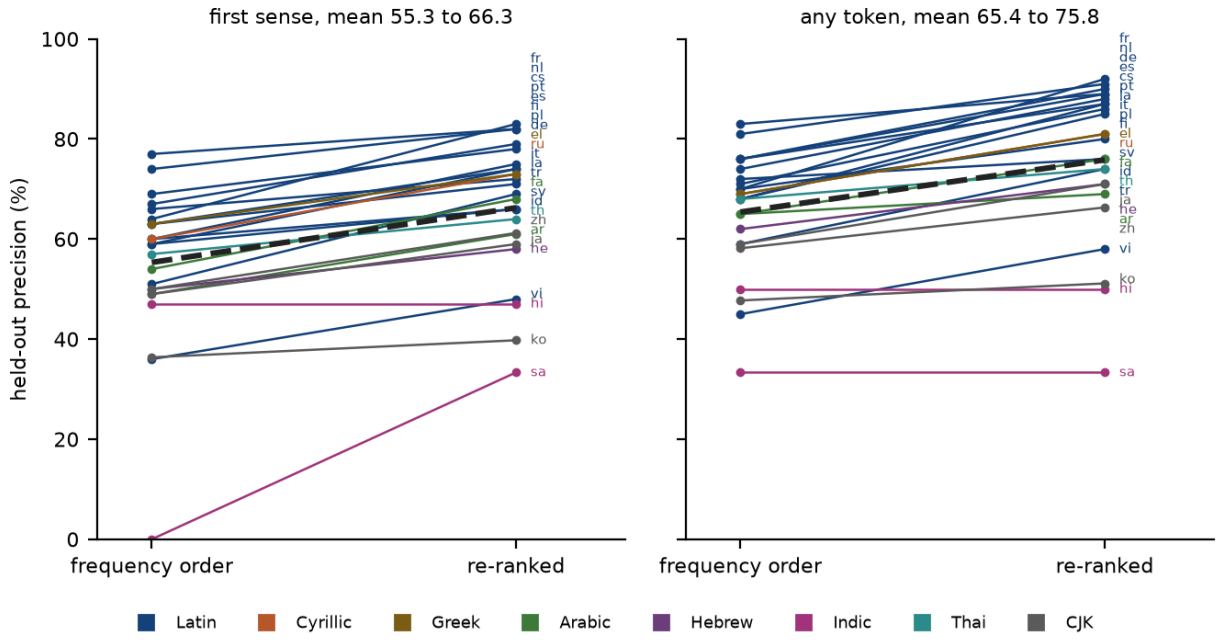


Figure 4: Before and after slopes per language on the held-out odd rows, first-sense and any-token precision; the dashed line joins the unweighted means over the 25 languages with held-out rows [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard: make figures].

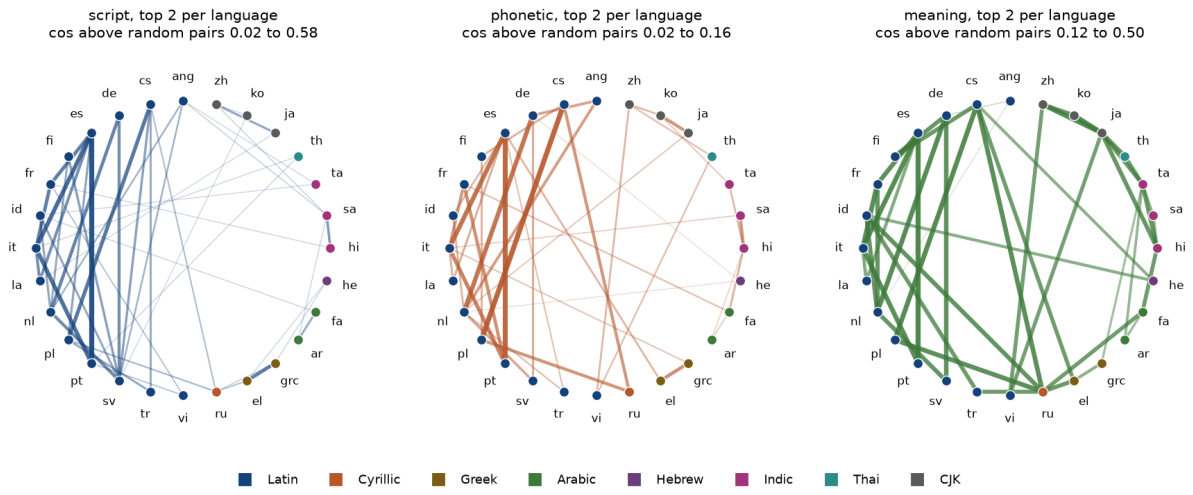


Figure 5: Language networks from the embeddings. For each pair of the 28 target languages, the edge weight is the mean cosine, under one view, between the two languages' first re-ranked translations of the same English word, over 600 English anchor words drawn with seed 7 from the 1,332 whose first translation exists in at least 20 languages. The edge weight is that mean minus a null baseline, the mean cosine over the same anchors with the second language's words shuffled into a derangement. Each language keeps its two strongest edges; a pair with fewer than two shared anchors gets no edge; width and opacity scale with the excess cosine within each panel [empirical: data/langgraph.json sha256 e9d1ca42, 2026-11-06, charizard: python3 papers/tools/langgraph.py derive --root ~/agfarms/data/polingual-embeddings/v1].

sha256 e9d1ca42, 2026-11-06, charizard: python3 papers/tools/langgraph.py derive]. Phonetic edges follow relatedness across scripts: 15 of 40 cross families, the strongest is Czech to Polish at 0.16 above the null, and Russian's two nearest languages by sound are Polish and Czech [empirical: data/langgraph.json sha256 e9d1ca42, 2026-11-06, charizard: python3 papers/tools/langgraph.py derive]. Meaning edges follow the encoder's shared space: Russian sits next to Czech and Polish at 0.46 above the null, and Japanese next to Korean at 0.48 [empirical: data/langgraph.json sha256 e9d1ca42, 2026-11-06, charizard: python3 papers/tools/langgraph.py derive].

The phonetic view puts the homophones *write* and *Wright* at cosine 1.00, and also *writer*, since four position bins average away a final syllable [empirical: data/neighbors_en_right.json sha256 763b1696, 2026-11-06, charizard: python3 papers/tools/langgraph.py neighbors --lang en --word right]. The script view finds the rhymes that look alike, *tight*, *night*, *fight* and *knight*, and none of the homophones.

6 Calibration

Every hyperparameter was chosen on one half of an evaluation set and is reported on the other half, Table 1 and Section 5.2. The Python replica of the app lookup gave 66.8% any-token precision on the full Tatoeba row for the lexicon of pull request 1198, where the Swift evaluation gave 67.5% [empirical: dataset v1 lexicon of PR 1198, 2026-11-06, charizard CPU: python3 apple/scripts/embed/pyeval.py lexicon, and data/swift_before.txt]; the replica skips the deck corpus layer, clitic stripping and NLTagger lemmas, which accounts for the 0.7 point gap. A rebuild of pull request 1198's lexicon with this pipeline in dump mode reproduces every line of every file. The Tatoeba references are noisy: they are the three English words that co-occur most with the target word, so a right sense that is less

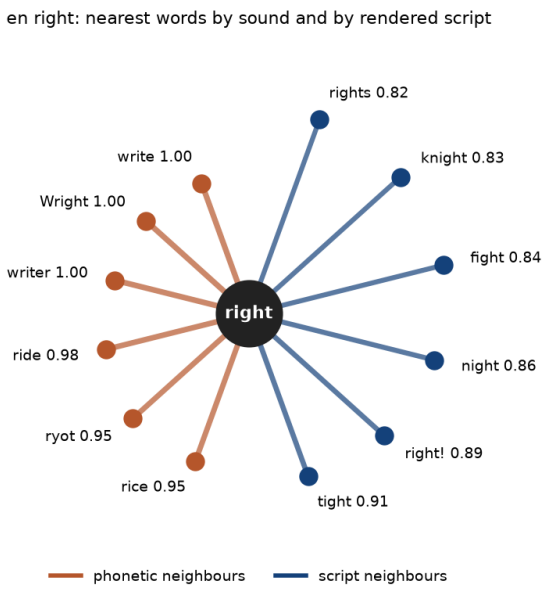


Figure 6: Homophone and confusable neighbours of English *right*: the six nearest English words by phonetic cosine, left, and by script CNN cosine, right, with the case variant *Right* removed; distance from the centre grows as the cosine falls [empirical: data/neighbors_en_right.json sha256 763b1696, 2026-11-06, charizard: python3 papers/tools/langgraph.py neighbors --lang en --word right].

frequent in Tatoeba counts as wrong. The error budget for the headline is the sampling error of a proportion near 0.68 over 2,325 held-out words, a standard error of 0.97 points, small next to the 10.3 point change [empirical: dataset v1 reports/rerank_report.json sha256 968b6767, 2026-11-06, charizard CPU: CUDA_VISIBLE_DEVICES= python3 apple/scripts/embed/rerank.py]. The audit covers 100 items, so its 31 point gap carries a standard error near 5.7 points [empirical: dataset v1 reports/audit_report.json sha256 c63af75e, 2026-11-06, python3 apple/scripts/embed/audit_sample.py, then python3 apple/scripts/embed/audit_judge.py].

7 Limitations

The English-to-target audit is a model audit; a human audit of the same 100 items would settle it. The lexicon induction scores in Table 8 use the Wiktionary candidate set as gold, so they share the known faults of dictionary-based evaluation: missing valid translations count as errors, and rare forms score lower [14, 15, 13]. The language networks of Figure 5 average over one seeded sample of 600 anchors, and a different seed moves the weaker edges; they describe this lexicon and are not a typology. The LaBSE language list, Table 10 of [19], includes Latin and omits Ancient Greek, Sanskrit and Old English, so meaning scores for those three lean on shared subwords. The phonetic view ignores tone, so Mandarin, Thai and Vietnamese homophones that differ by tone collapse. Swedish and Hebrew have 5,054 and 2,872 WikiPron entries, too few for a good grapheme model: G2P R@10 is 5.5% and 13.5% [empirical: dataset v1 reports/phonetic_report.json sha256 5273260e, 2026-11-06, charizard RX 7600 ROCm 6.4 torch 2.9.1: python3 apple/scripts/embed/phonetic.py]. The script CNN trains on weight changes of one font family per script, so it learns little about handwriting or other typefaces. Tamil has no usable Tatoeba-aligned references, since the alignment tokenizer splits Tamil vowel signs. The joint projection improves P@10 but not P@1 over meaning, and the re-ranker selected

the meaning cosine; the joint vectors ship in the dataset and stay out of the app.

Licenses. Wiktionary text is CC BY-SA 4.0, https://en.wiktionary.org/wiki/Wiktionary:Copyrights; kaikki.org serves its extraction under the Wiktionary licences, https://kaikki.org/dictionary/; WikiPron code is Apache 2.0 and its scraped data keeps the Wiktionary terms, https://github.com/CUNY-CL/wikipron. The re-ranked lexicon and dataset carry CC BY-SA 4.0. Tatoeba sentences are CC BY 2.0 FR, https://tatoeba.org/en/terms_of_use. Noto fonts are SIL OFL 1.1, https://github.com/notofonts/noto-fonts. LaBSE is Apache 2.0, https://huggingface.co/sentence-transformers/LaBSE. PanPhon is MIT, https://github.com/dmort27/panphon/blob/master/LICENSE.txt.

References

[1] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. 2013. arXiv: 1301.3781 [cs.CL].

[2] Tomas Mikolov et al. *Distributed Representations of Words and Phrases and their Compositionality*. 2013. arXiv: 1310.4546 [cs.CL].

[3] Piotr Bojanowski et al. *Enriching Word Vectors with Subword Information*. 2016. arXiv: 1607.04606 [cs.CL].

[4] Tomas Mikolov, Quoc V. Le, and Ilya Sutskever. *Exploiting Similarities among Languages for Machine Translation*. 2013. arXiv: 1309.4168 [cs.CL].

[5] Peter H. Schönemann. "A Generalized Solution of the Orthogonal Procrustes Problem". In: *Psychometrika* 31 (1966), pp. 1–10. DOI: 10.1007/BF02289451.

[6] Chao Xing et al. "Normalized Word Embedding and Orthogonal Transform for Bilingual Word Translation". In: *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2015, pp. 1006–1011. DOI: 10.3115/v1/N15-1104.

[7] Samuel L. Smith et al. *Offline bilingual word vectors, orthogonal transformations and the inverted softmax*. 2017. arXiv: 1702.03859 [cs.CL].

[8] Mikel Artetxe, Gorka Labaka, and Eneko Agirre. "Learning principled bilingual mappings of word embeddings while preserving monolingual invariance". In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2016, pp. 2289–2294. DOI: 10.18653/v1/D16-1250.

[9] Mikel Artetxe, Gorka Labaka, and Eneko Agirre. "A robust self-learning method for fully unsupervised cross-lingual mappings of word embeddings". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2018, pp. 789–798. DOI: 10.18653/v1/P18-1073.

[10] Alexis Conneau et al. *Word Translation Without Parallel Data*. 2017. arXiv: 1710.04087 [cs.CL].

[11] Georgiana Dinu, Angeliki Lazaridou, and Marco Baroni. *Improving zero-shot learning by mitigating the hubness problem*. 2014. arXiv: 1412.6568 [cs.CL].

[12] Sebastian Ruder, Ivan Vulić, and Anders Søgaard. "A Survey of Cross-lingual Word Embedding Models". In: *Journal of Artificial Intelligence Research* 65 (2019), pp. 569–631. DOI: 10.1613/jair.1.11640.

[13] Goran Glavaš et al. "How to (Properly) Evaluate Cross-Lingual Word Embeddings: On Strong Baselines, Comparative Analyses, and Some Misconceptions". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 710–721. DOI: 10.18653/v1/P19-1070.

[14] Yova Kementchedjhieva, Mareike Hartmann, and Anders Søgaard. "Lost in Evaluation: Misleading Benchmarks for Bilingual Dictionary Induction". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3334–3339. DOI: 10.18653/v1/D19-1328.

[15] Paula Czarnowska et al. *Don't Forget the Long Tail! A Comprehensive Analysis of Morphological Generalization in Bilingual Lexicon Induction*. 2019. arXiv: 1909.02855 [cs.CL].

[16] Anders Søgaard, Sebastian Ruder, and Ivan Vulić. "On the Limitations of Unsupervised Bilingual Dictionary Induction". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2018, pp. 778–788. DOI: 10.18653/v1/P18-1072.

[17] Ivan Vulić, Sebastian Ruder, and Anders Søgaard. *Are All Good Word Vector Spaces Isomorphic?* 2020. arXiv: 2004.04070 [cs.CL].

[18] Mikel Artetxe and Holger Schwenk. "Massively Multilingual Sentence Embeddings for Zero-Shot Cross-Lingual Transfer and Beyond". In: *Transactions of the Association for Computational Linguistics* 7 (2019), pp. 597–610. DOI: 10.1162/tacl_a_00288.

[19] Fangxiaoyu Feng et al. *Language-agnostic BERT Sentence Embedding*. 2020. arXiv: 2007.01852 [cs.CL].

[20] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3980–3990. DOI: 10.18653/v1/D19-1410.

[21] Nils Reimers and Iryna Gurevych. "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 4512–4525. DOI: 10.18653/v1/2020.emnlp-main.365.

[22] Christian M. Meyer and Iryna Gurevych. "Wiktionary: A new rival for expert-built lexicons? Exploring the possibilities of collaborative lexicography". In: *Electronic Lexicography*. Oxford University Press, 2012, pp. 259–292. DOI: 10.1093/acprof:oso/9780199654864.003.0013.

[23] Tatu Ylonen. "Wiktextract: Wiktionary as Machine-Readable Structured Data". In: *Proceedings of the Thirteenth Language Resources and Evaluation Conference (LREC 2022)*. European Language Resources Association (ELRA), 2022, pp. 1317–1325. DOI: 10.63317/5cx2hzmhn2ef.

[24] Kyle Gorman et al. "The SIGMORPHON 2020 Shared Task on Multilingual Grapheme-to-Phoneme Conversion". In: *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*. Association for Computational Linguistics, 2020, pp. 40–50. DOI: 10.18653/v1/2020.sigmorphon-1.2.

[25] Pierluigi Cassotti, Jacob Lee Suchardt, and Domenico De Cristofaro. "WikIPA: Integrating WikiPron and Lingua Libre for Multilingual IPA Transcription". In: *Proceedings of the Fifteenth Language Resources and Evaluation Conference (LREC 2026)*. European Language Resources Association (ELRA), 2026, pp. 8465–8475. DOI: 10.63317/2am4iw3bfhjb.

[26] Jörg Tiedemann. *The Tatoeba Translation Challenge – Realistic Data Sets for Low Resource and Multilingual MT*. 2020. arXiv: 2010.06354 [cs.CL].

[27] Naman Goyal et al. "The Flores-101 Evaluation Benchmark for Low-Resource and Multilingual Machine Translation". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 522–538. DOI: 10.1162/tacl_a_00474.

[28] Elizabeth Salesky, David Etter, and Matt Post. *Robust Open-Vocabulary Translation from Visual Text Representations*. 2021. arXiv: 2104.08211 [cs.CL].

[29] Phillip Rust et al. *Language Modelling with Pixels*. 2022. arXiv: 2207.06991 [cs.CL].

[30] Frederick Liu et al. *Learning Character-level Compositionality with Visual Features*. 2017. arXiv: 1704.04859 [cs.CL].

[31] Yuxian Meng et al. *Glyce: Glyph-vectors for Chinese Character Representations*. 2019. arXiv: 1901.10125 [cs.CL].

[32] Vilém Zouhar et al. *PWESuite: Phonetic Word Embeddings and Tasks They Facilitate*. 2023. arXiv: 2304.02541 [cs.CL].

[33] Taraka Rama. *Siamese convolutional networks based on phonetic features for cognate identification*. 2016. arXiv: 1605.05172 [cs.CL].

[34] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. *Representation Learning with Contrastive Predictive Coding*. 2018. arXiv: 1807.03748 [cs.LG].

[35] Ting Chen et al. *A Simple Framework for Contrastive Learning of Visual Representations*. 2020. arXiv: 2002.05709 [cs.LG].

[36] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG].

[37] Lee R. Dice. "Measures of the Amount of Ecologic Association Between Species". In: *Ecology* 26 (1945), pp. 297–302. DOI: 10.2307/1932409.

A Glossary

lexicon https://github.com/bucket-foundation/bucket-foundation/blob/dev/apple/scripts/translate/build_lexicon.py
The per-language Polingual word list: each headword with up to four English meanings, a gloss, a romanization, form links and English-to-target lists; see Definition 3.1.

modality https://github.com/bucket-foundation/bucket-foundation/blob/dev/apple/scripts/embed/common.py
One of the four word views embedded here: script, character, phonetic, meaning; see Definition 3.2.

joint score https://github.com/bucket-foundation/bucket-foundation/blob/dev/apple/scripts/embed/joint.py
A weighted sum of per-modality cosines, or the cosine of a learned projection of all four vectors; see Definition 3.3.

precisionhttps://github.com/bucket-foundation/bucket-foundation/blob/dev/apple/BucketKit/Tests/BucketKitTests/TranslateEvalTests.swift
Judged answers marked right divided by judged answers; see Definition 3.4.

B Lean listings

`lean/Bucket/Embed.lean` holds a counterpart of each Preliminaries definition: `Lexicon` for Definition 3.1, `Modality` and `Modality.cos` for Definition 3.2, `jointScore` and `jointOf` for Definition 3.3, and `precisionPct` for Definition 3.4. It proves that the app shows at most three senses, that the dot product and so every modality cosine and joint score is symmetric, that the meaning-only joint score is the meaning cosine, that precision is at most 100 when right answers do not exceed judged ones, and that filtering senses never lengthens a list. Vectors are integer lists, so the proofs cover the algebra and leave floating point aside.

```
structure Lexicon (W E : Type) where
  english : W -> List E
  gloss : W -> List E
  targets : E -> List W
theorem Lexicon.shown_length_le (lex : Lexicon W E) (w : W) :
    (lex.shown w).length <= 3
theorem dot_comm : forall a b : List Int, dot a b = dot b a
def Modality (W : Type) := W -> List Int
def Modality.cos (m : Modality W) (a b : W) : Int := dot (m a) (m b)
theorem Modality.cos_symm (m : Modality W) (a b : W) :
    m.cos a b = m.cos b a
def jointScore (w c : List Int) : Int := dot w c
def jointOf (w : List Int) (ms : List (Modality W)) (a b : W) : Int :=
  jointScore w (ms.map fun m => m.cos a b)
theorem jointOf_symm (w : List Int) (ms : List (Modality W)) (a b : W) :
    jointOf w ms a b = jointOf w ms b a
theorem jointScore_meaning_only (s p c m : Int) :
    jointScore [0, 0, 0, 1] [s, p, c, m] = m
def precisionPct (right judged : Nat) : Nat := right * 100 / judged
theorem precisionPct_le (right judged : Nat) (h : right <= judged) :
    precisionPct right judged <= 100
theorem keepSenses_length (keep : a -> Bool) (senses : List a) :
    (keepSenses keep senses).length <= senses.length
```

`lake build` output, captured by `make lean`. This block is a verbatim transcript.

```
cd lean && lake build
✓ [2/4] Built Bucket.Embed (1.1s)
✓ [3/4] Built Bucket (209ms)
Build completed successfully (4 jobs).
```